Smart Water Filter Base

Design Document

CSE 123A - Winter 2026

Group 8

Contributors:

Logan Bossuwe
Dev Chodavadiya
Shriyan Gote
Reid Graham
Sam Lai
Pieter Nauwelaerts

March 13, 2026

Executive Summary

This design document covers the creation of a our Smart Water Filter Base, which is a device that monitors the weight of water in a pitcher to determine the remaining water level and notify users when it is low.

Shared environments, including roommates, families, and offices, often rely on a single water filter. This can often lead to frustration if users frequently discover that the container is empty when they try to use it. This causes inconvenience, wasted time, and frustration.

Our proposed solution is a smart base that is placed under the filter and continuously measures the weight. This weight is converted into a fullness level and sent to users through a mobile app, notifying them if it gets low.

The system is composed of hardware, software, and web components.

Hardware:

- Load cell for measuring weight
- HX711 amplifier for sensor communication
- WiFi-enabled microcontroller for processing and communication

Software:

- Microcontroller firmware for reading sensor data and sending notifications
- HTTP server for communication between the microcontroller and the mobile app
- Mobile app for displaying water level and notifications

The Smart Water Filter Base is a simple, practical solution for monitoring water filter levels. It is easy to use and install, working with existing water filter pitchers without requiring any modifications. By providing real-time updates on water levels, it helps users avoid the inconvenience of discovering an empty pitcher when they need water.

Contents

1 Introduction

1.1 Problem statement

Living with roommates can sometimes mean your filtered water device is empty when you need water. This can be very frustrating and leave you needing to drink tap water while other roommates get to drink cold refrigerated and filtered water.

1.2 Need Statement

We want a way to know if the container is out of water when we need water.

- Avoid having no water left in addition to everybody using the water filter, not knowing there is no water left.
- Need a way to know the current water level of the pitcher so that it isn't empty without everyone knowing it's empty.
- We need to know if the container is empty, in order to not run out of clean fresh water.

1.3 Goal Statement

- Keeping water level known at all times, while informing everybody in the household if the water level runs too low.
- Create a system to inform users if the container is empty.

If the container is empty, inform users.

1.4 Personas and Users

This is intended for shared households that rely on a shared filtration water container.

- College Roommates
  - John Doe, 20 years old, Student at UCSC. John lives in a dorm with two other roommates, and they all rely on a shared water filter for drinking water. Occasionally, John comes out to get water only to find that the water is empty. This creates frustration and delays as John has to choose between missing the bus to refill water or getting to class on time while being thirsty.
- Family
  - Jane Smith, 39 years old, working mother. Jane lives with her husband and two children. Jane gets home after a long, exhausting day of work and is feeling thirsty from the drive home. She goes to the shared water filter to find that it is completely empty. Jane feels frustrated with her stay-at-home husband and two children for not refilling the water filter. This causes her to lash out and yell at them until they all cry, creating a hostile environment in the household.

- Office
  - Joe Micheal, 30 years old, works with Excel spreadsheets. He has very limited time to refill his mug, and his boss, George, never refills the water filter. As a result, Joe's efficiency has substantially decreased at his desk.

1.5 Research into Existing Designs and Products

To understand where our design fits, we researched three existing products that partially meet the same need.

1.5.1 Withings Body Smart

The Withings Body Smart is a consumer Wi-Fi scale that measures weight, body fat, muscle mass, and water percentage using strain gauge load cells and bioelectrical impedance analysis. It syncs data automatically to the Withings smartphone app, where users can track trends and set goals. While it offers a polished user experience, the system is entirely closed—users must rely on Withings' proprietary app and cloud service, with no way to self-host data or customize the interface.

1.5.2 Wyze Scale Ultra

The Wyze Scale Ultra takes a similar approach but adds a 4.3-inch color display directly on the scale, so users can view 13 body composition metrics without needing their phone nearby. It supports Wi-Fi syncing to the Wyze app and integrates with third-party fitness platforms like Apple Health and Google Fit. Like the Withings, it is a closed system that cannot be modified or repurposed beyond its intended body composition use case.

1.5.3 Hackster.io ESP32 MQTT Weight System

On the DIY side, a Hackster.io project by The Embedded Things implements an ESP32-based weight measurement system using an HX711 amplifier and load cell. It features MQTT communication for real-time data streaming, multi-sample averaging for noise reduction, and automatic tare on startup. This project demonstrates strong signal processing but requires an external MQTT broker and separate dashboard software, adding complexity that our self-hosted approach avoids.

1.5.4 Where Our Design Fits

Our project fills a gap between these existing solutions. The commercial scales offer polished experiences but are closed systems tied to proprietary apps. The Hackster.io project is open-source and uses similar hardware, but depends on external services for data display. Our design combines the low cost and openness of a DIY build with a fully self-hosted web interface on a microcontroller that provides real-time weight display, historical logging, and alerts—all without needing external apps or cloud services.

1.6 Sustainability Statement

Our project promotes sustainable water consumption habits in shared households. Without knowing the current water level, users often open the fridge only to find an empty filter, leading them to either waste time waiting for it to refill or even use one-time plastic bottles. By providing real-time water level monitoring and low-level alerts, our system encourages refilling and reduces the likelihood of users turning to less sustainable alternatives. Over time, this helps households maintain consistent use of their reusable water filter rather than abandoning it out of frustration.

2 Design

2.1 Design Objective

Through the goal statement, it is needed that water level is known at all times to the user. To approach the goal, two parts needed being what hardware is needed to get accurate readings and what software is needed to communicate the information to the user.
Hardware We will be using weight as the solution to getting accurate readings for water levels. To measure weight, the hardware would be a load cell, amplifier and a microcontroller. For each hardware, there are design choices made based off of the product.

2.1.1 Load Cell + HX711 Amplifier + Microcontroller

Here are the choices made for the load cell we chose:

- Using a 20kg Load Cell, it gives sufficient room for most commerical water filter pitchers
- HX711 is a high resolution 24-bit analog to digital converter. The amplifier is specifically built for load cells.
- Microcontroller will have functionalities of Wifi, 2 Pins, GND, 3.3V VCC as essential components.

2.2 Aesthetic Prototype

Our current prototype consists of two 3D-printed circular base plates sandwiching a 75mm bar-type strain gauge load cell. Each plate is 110mm in diameter and 8mm thick, printed in PLA/PETG. The top plate has countersink pockets on the bottom face for M4 bolts that secure it to one end of the load cell, and the bottom plate attaches to the other end, sitting flat on the surface. The load cell wires route out to the HX711 amplifier and microcontroller on a nearby breadboard.

2.2.1 Exploded View

Figure 1 shows the exploded view of the load cell assembly with all components labeled.

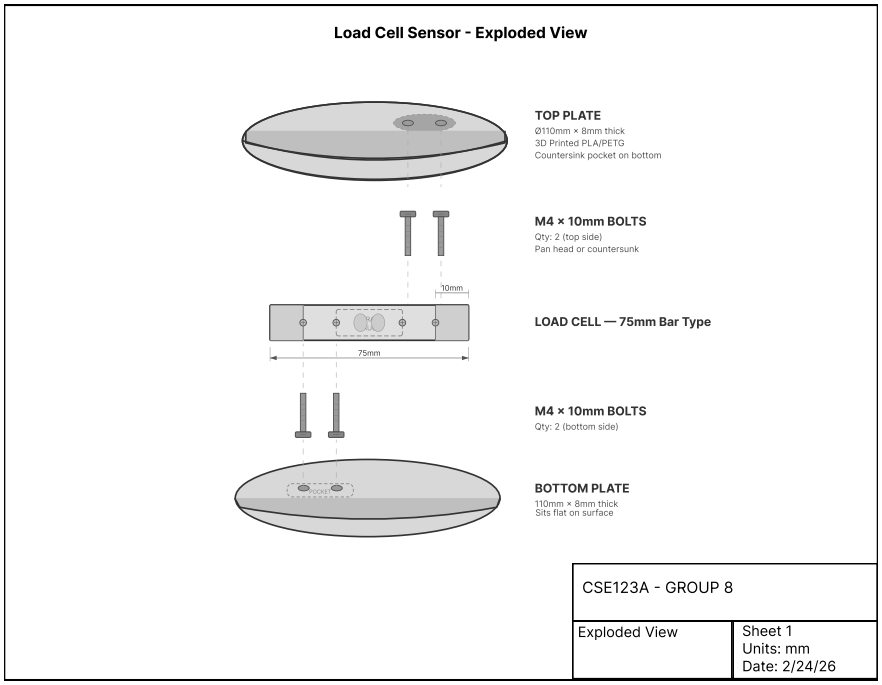


Figure 1: Exploded view of the load cell assembly showing top plate, bottom plate, load cell, and M4 hardware.

2.2.2 Base Plate Schematic

Figure 2 shows the design schematic of the base plate with dimensions and screw hole placement.

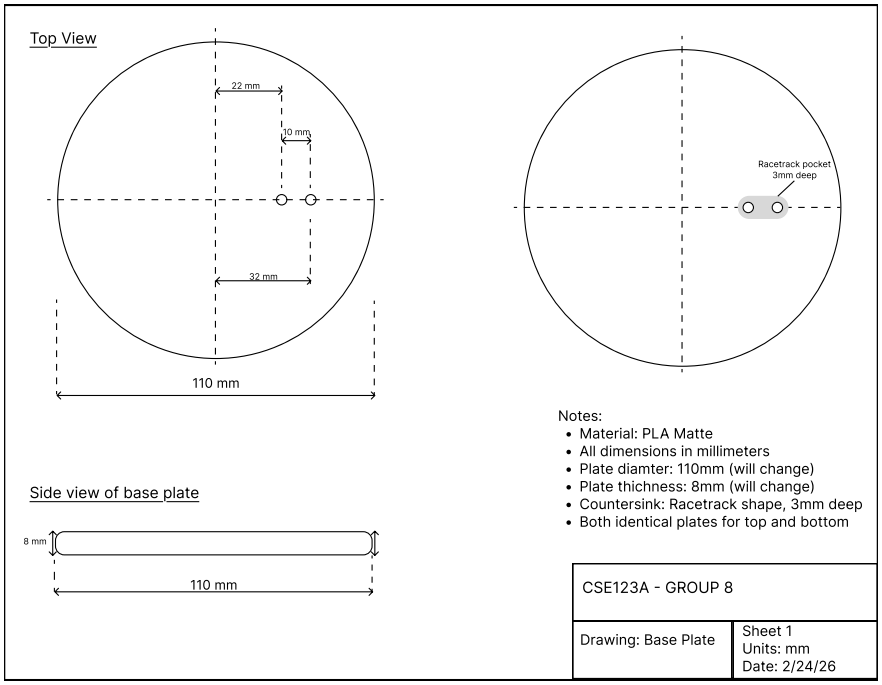


Figure 2: Design schematic of the base plate.

2.2.3 Physical Prototype

Figure 3 shows the assembled prototype from a side view, with the load cell mounted between the two plates and wired to the breadboard.

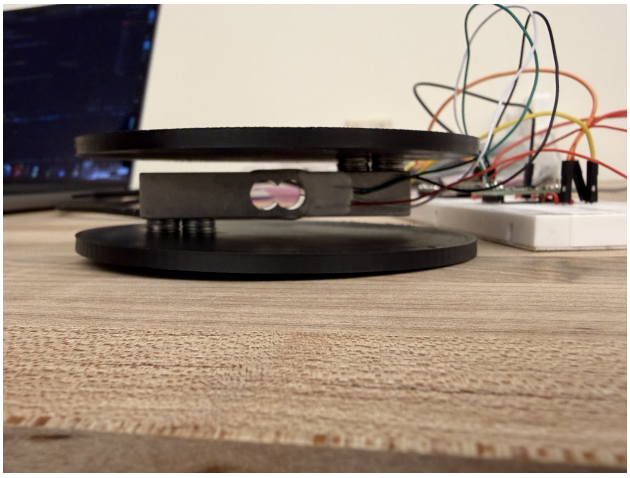


Figure 3: Side view of the assembled prototype showing the two base plates, load cell, and wiring to the microcontroller.

2.3 Design for Manufacture

2.3.1 3D-Printed Base Plates

The top and bottom plates are 3D-printed in PLA at a 0.2mm layer height. Each plate is 110mm in diameter and 8mm thick with M4 screw holes and racetrack-shaped countersink pockets. The plates went through multiple print iterations to resolve a flatness issue where the bottom surface was slightly slanted, which affected the stability of the load cell. This was fixed by adjusting bed leveling and print orientation to ensure a flat bottom surface.

2.3.2 Load Cell Assembly

The 75mm bar-type strain gauge load cell is sandwiched between the two plates using four M4 x 10mm bolts—two through the top plate into one end of the load cell, and two through the bottom plate into the other end. This creates a cantilever arrangement where weight applied to the top plate causes measurable strain on the load cell. The assembly requires only an M4 hex key and can be taken apart and reassembled in a few minutes.

2.3.3 Electronics

The microcontroller and HX711 amplifier are currently mounted on a breadboard alongside the prototype. The HX711 connects to the load cell via four wires (E+, E-, A+, A-) and communicates with the microcontroller over two GPIO pins (data and clock). Power is supplied to the microcontroller via USB. The breadboard setup allows for easy debugging and rewiring during development, with the intent to move to a more permanent soldered configuration in the final version.

2.4 Block Diagrams

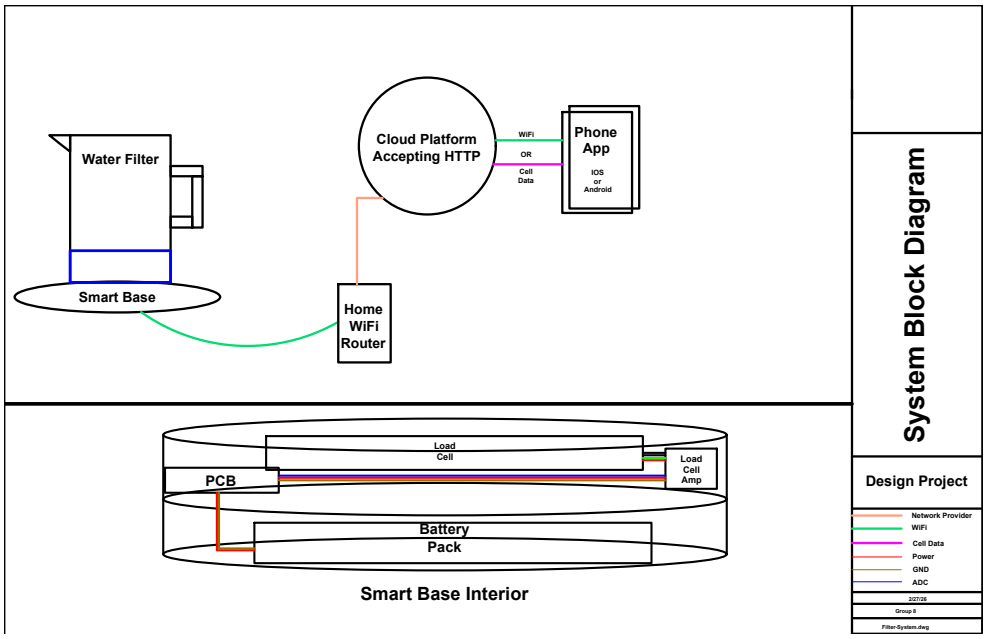


Figure 4: The system block diagram represents the pieces of the device and how they are interconnected.

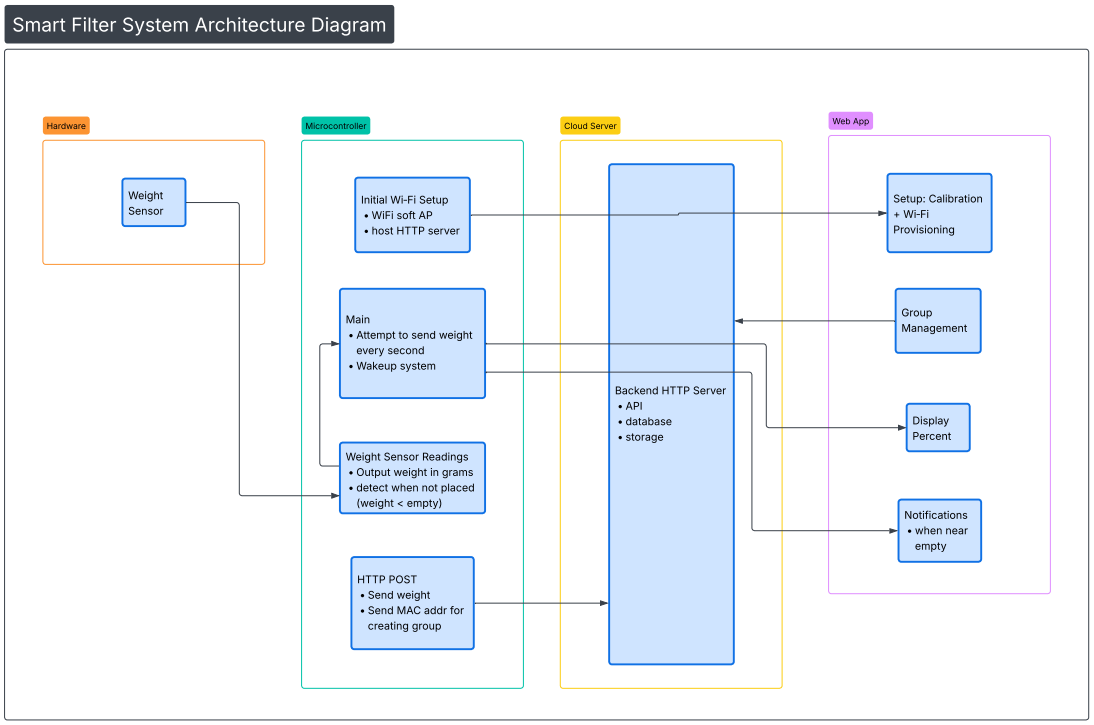


Figure 5: The basic architecture of our project, including the modules and submodules of hardware and software, along with how they interact and communicate with each other.

2.5 Wiring Diagrams

TODO

2.6 Microcontroller WiFi Provisioning

In order for the microcontroller to connect to the home network, the user must be able to send the WiFi credentials to the device. This is called WiFi provisioning. In our design, the MCU hosts a WiFi soft access point, an HTTP server, and an HTML page that allows the user to input the WiFi credentials. Once it is connected to the home network, it will save this information and can communicate with the web application.

2.6.1 First Boot

When the device is first turned on, it will attempt to connect to the last known WiFi network using credentials from non-volatile storage. If the connection is successful, the device will operate normally and send data to the web application. If the connection fails (e.g. because the credentials are incorrect or the network is unavailable), the device will enter provisioning mode.

2.6.2 Soft Access Point

In provisioning mode, the MCU will create its own WiFi soft access point with a unique name (e.g. "Smart Filter Setup"). On that network, it will host an HTTP server on a specific IP address along with an HTML page. The user can then follow the instructions on the device to connect to this network using their phone or computer, scan a QR code to access the HTML page, and enter their WiFi credentials. Once the user submits the form with their credentials, the device will attempt to connect to the specified WiFi network. If the connection is successful, it will save the credentials to non-volatile storage for future use and exit provisioning mode. If the connection fails, it will display an error message and remain in provisioning mode until valid credentials are provided.

2.7 HTTP Functionality

The Initial prototype will send data via HTTP POST requests. The microcontroller is capable of creating these requests and sending them via the WiFi connection.This is a simple way of updating our web server with any new weight information. HTTP POST requests are simple and useful at doing what we need for this initial prototype.

2.7.1 Request Format

The request sends only a single line. The line is only the current weight measured by the sensor at the time of sending. Using a formatted string, the following is sent in the HTTP Packet: "weight=x.yz" Two points after the decimal are sent for accuracy for lower volume containers.

The sensor data is converted to grams earlier in the code, so with an already calculated value all the HTTP code needs to do is package it and send it as a POST request. The post data is sent using the built in send_http_post() function. This function is a part of the microcontroller HTTP utilities. For production, HTTP data would be sent using C sockets.

2.7.2 Receiving Data

On the server side, the POST data is received and using the expected format, decoded and sent to storage using Supabase. The data storage method and how the website pulls from the server will be discussed in their own section.

2.8 Web Application

The web application is the user-facing interface for the smart water-level monitor. The design goal is to provide a clear real-time estimate of remaining water in the container, allow user calibration for different pitcher sizes, and send refill notifications when the water level becomes low.

2.8.1 Initial Design

In its most simple form it must be able to display static data on a web page. We have created a .html file with this basic information, shown in the following figure.

Figure 6: Output of Initial HTML Code

From here there are a few changes that can be made. These changes follow the line of next steps to test this code and our designs.

- Dynamically updating data with a controller (Proof of concept testing w/ simple text)
- Sending and interpreting sensor data
- Graphic interface changes
- Sending push notifications to a phone

In order to test these things we will create a local server interface using one of our computers that hosts this data. Then we will use a microcontroller to send HTTP PUT requests. Finally, using the received data to change what gets displayed.

2.8.2 Current Design

The current design is a React application deployed on Vercel, with a Supabase backend for data storage. The frontend displays the latest water level reading as a percentage, along with a visual representation of the water level in the pitcher. Users can calibrate the system by setting empty and full reference points, which allows for accurate level estimation regardless of pitcher size or sensor placement. The application also includes functionality to send push notifications to users when the water level falls below a certain threshold.

2.8.3 System Architecture

The deployed architecture has three coordinated layers:

- **Frontend dashboard:** Displays the latest reading, computes estimated level percentage, and status indicators.
- **Vercel backend:** Receives sensor uploads and notification subscriptions.
- **Supabase Storage:** Stores water readings, calibration parameters, and browser push subscription tokens.

The frontend was designed using the following languages: HTML, CSS, and JavaScript. It utilizes a React framework for building user interfaces. The backend is implemented on Vercel, written in JavaScript (Node.js), with API endpoints for data ingestion and notification management. We are using Supabase, which provides a PostgreSQL database for persistent storage and real-time capabilities.

2.8.4 Frontend Design and User Experience

The current dashboard is minimal and task-focused. Core interface elements are:

- A large water-level visualization (pitcher graphic with animated fill height).
- A numerical percentage readout for quick interpretation.
- A water-present/empty status indicator.
- Metadata showing last update timestamp, measured weight, and battery voltage (when available).

Users can quickly assess whether refill action is needed without interpreting raw sensor values.

2.8.5 API Endpoints

The application has two primary API endpoints:

- **POST /api/ingest:** Receives sensor readings (device ID, weight, battery) from the microcontroller and stores the data in Supabase. If the weight is below a configured threshold, triggers a low-water notification to registered users.
- **POST /api/register-token:** Registers browser push subscriptions from the frontend and saves them in the database.

Sensor data is sent to the ingestion endpoint with an API key, validated, and inserted into a table on Supabase. The dashboard reads the newest value from Supabase on a polling interval and renders the latest state. The token registration endpoint enables user devices to register for alert notifications. When invoked, it associates a device with a user account and stores the device token in a table on Supabase for later use in push notification delivery.

2.8.6 Supabase Tables

Supabase serves as persistent storage for both backend APIs and the frontend dashboard state. The ingestion and token-registration endpoints write data to Supabase using a service-role key, while the frontend dashboard queries the same tables to render the latest readings and calibration settings.

The current deployment uses the following tables:

- `water_readings` — Stores time-series sensor uploads from the microcontroller. Each row represents one sample and includes the device name, weight in grams, and battery voltage in millivolts (e.g., `device_id`, `weight_g`, `battery_mv`) with a timestamp (`created_at`).
- `notification_tokens` — Stores browser/device push tokens used for low-water alerts. Tokens are inserted or upserted on registration and are later queried by the ingest pipeline when a low-water event occurs.
- `calibration` — Stores user calibration references for level estimation. The dashboard upserts a single calibration record (using `id` constant) containing empty and full reference values (`empty`, `full`), then reads the latest entry when computing displayed percentage.

2.8.7 Calibration

To support different pitcher geometries and sensor offsets, the application includes user calibration controls:

- **Calibrate empty:** Stores the current reading as the empty reference.
- **Calibrate full:** Stores the current reading as the full reference.
- **Reset calibration:** Clears stored values and falls back to default constants.

The level estimate is computed by linearly mapping measured weight between empty and full references, then clamping the result between 0% and 100%. This makes the UI behavior predictable while still allowing practical adjustments.

2.8.8 Push Notifications

The push notification feature is designed as an event-driven subsystem that alerts users only when refill action is needed. The implementation uses standard Web Push protocols with VAPID authentication and is integrated into the ingest API used for sensor data.

1) Subscription Registration When the dashboard is loaded, the browser UI has a button that the user can click on to request notification permission. Clicking on the button triggers a pop up that asks the user to allow/disallow notifications for the website. If permission is granted, a service worker is registered and a Push API subscription is created (or reused if it already exists). The subscription object is sent to the backend endpoint and stored in the `notification_tokens` table in Supabase.
This registration path has two design goals:

- maintain a durable list of reachable user devices
- avoid duplicate subscriptions by upserting on token value.

2) Alert Trigger Condition Sensor readings are received by the ingestion endpoint and stored in `water_readings` table. For each new reading, the backend computes current and previous water-level percentages and checks for a low-level threshold crossing. The low-water threshold is currently set to 20%. This design prevents repeated notifications while the level remains continuously low.

3) Delivery If the trigger condition is true, the backend loads all stored subscriptions and sends a Web Push payload to each valid endpoint. The payload includes a:

- notification title
- notification body
- structured data (event type and level percentage) for client-side handling.

The service worker receives the push event, displays the notification, and handles click interaction by focusing an existing app tab or opening the web page.

2.9 Web Application Testing

The web app test file is designed to check the main user-facing behavior of the React application in a clear and controlled way. The tests render the app with mock data so we can verify behavior without needing live hardware or external services. The test file is organized under the test directory within the source directory, along with the file `sampleWaterValues.js` that contains mock data. The test file uses the React Testing Library to render components and simulate user interactions, and Vitest for mocks, assertions, and test organization.

2.9.1 Basic UI Rendering

This section verifies that the interface renders correctly in both empty and normal states. It checks the message shown when no reading exists, then confirms that water-level data, weight, and battery metadata are displayed when readings are available. It also checks that the displayed percentage matches the expected calculation.

2.9.2 Calibration Actions and Edge Cases

This section tests user interaction with calibration controls. The test suite clicks the calibrate empty, calibrate full, and reset buttons, then verifies that the expected calibration upsert operations are made. This section verifies calibration logic for both standard and edge conditions. The tests confirm that custom empty/full calibration values are used, and that percentage output is clamped correctly. Readings below empty are shown as 0%, readings above full are shown as 100%, and invalid ranges where full is less than or equal to empty also resolve to 0%.

2.9.3 Push Notification Behavior

This section tests notification flows with mocked browser APIs. It includes unsupported-browser behavior, denied permission behavior, and successful permission behavior. In the successful case, the test verifies service worker registration and push subscription setup.

2.9.4 API-Related Checks

This section covers API behavior that is directly tested. Specifically, when notifications are enabled successfully, the app is verified to send a POST request to the token registration endpoint. Supabase reads and calibration writes are mocked so request behavior and state changes can be checked without live backend dependencies.

2.10 HX711

In order to get readings for weight from the load cell, we use the HX711 Amplifer that converts analog to digital converter.

2.10.1 SCK

- SCK is the clock line.
- Controls when bits are shifted out of DOUT, each pulse moves the next data bit to the output.
- SCK LOW-¿HIGH - Shifts out one bit
- SCK being HIGH for some time - Power down mode
- LOW after power down - Sensor wakes up and converts data

2.10.2 DOUT

- DOUT is the data output that sends 24-bit measurement to microcontroller
- DOUT when HIGH - HX711 currently converting data not ready
- DOUT when LOW - Data is ready to read

2.10.3 Pins

- VCC - power supply around 2.7-5.5V
- GND - Ground
- E+ - Positive excitation voltage
- E- - Negative excitation voltage
- A+ - Positive signal input, Chanel A

- A- - Negative signal input, Chanel A
- B+ - Positive signal input, Chanel B
- B- - Negative signal input, Chanel B

2.10.4 Read Data

The HX711 Sensor is a 24-bit analog to digital converter. To read data, DOUT gives raw 24bits that need be read on pulses. Since SCK is a pulse that will shift 1 bit each time, you need to do this 24 times for the full number. Additionally, there are extra pulses needed for the HX711 to select chanel. Create a delay when waiting for DOUT to go low again, sometimes it might stay high.

2.10.5 Average Sampling

Sampling without averaging can lead to lots of noise. Average sampling allows the accuracy of weight to be more accurate.

2.10.6 Calibration

The Sensor gives raw data, as counts. From testing, have a hard coded number that when using to calculate the grams from the raw data, divide raw by the hard coded number. Use known weights to calibrate accurately.

2.10.7 Tare

For handling offset, tare function will take the current weight to zero. Whenever an object is placed later, the weight displays is sampling - offset. Sampling gives total weight, offset allows to remove the pre-existing weight to display the objects true weight.

2.10.8 Debug

Print statements every second each signal/number:

- SCK
- DOUT
- Raw Sample
- Offset
- Grams

Allows to bugs/problems code or hardware.

2.11 Testing HX711

To test the code written for this project, we defrost had to create a simulated version of the sensor. Regardless of what the sensor reports we want to make sure that all of our math and interpretation of incoming data is correct. To do this, the functions were adapted to either take input from GPIO pins, or from a specific test file.

2.11.1 Test File

Inside the test file are adapted versions of each function used by the main HX711 file. These include the following:

- hx711_set_sck
- hx711_get_dout
- read_raw
- get_raw_weight
- tare
- get_grams

With each of these functions, the test file simulates reading data from the sensor.

2.11.2 Simulated Data

Using some set values defined at the top of the file including a hardcoded offset and scaling factor, the test file first sets the weight and tares it. Then based on a random number between 0 and 2, it sends either a 0, small weight, or large weight value.

When the test file calls to the read weight functions, some small noise of $\pm 1g$ is used simulate the variability of the sensor reading. The sensor sends data in 24 bit samples, so the test code also includes the 24 bit shifting. Once the simulated data is created, the main function compares it to the expected result. If it's within one gram of the expected low high or zero value, then the value passes. This tolerance is arbitrary for the testing and will be different for the production code.

3 Evaluation

3.1 Functional Prototype

The prototype will consist of weight sensor readings sent to HTTP server via POST. Webapp will be shown using fake data to demonstrate handling of data in grams with notifications
Hardware Setup The plates used above and below the load cell are made for functional prototype purposes to fit the need of measure weight readings rather than handling various water filter pitchers. Temporary HTTP server made to display and mimick how data is shown/handled.

Webapp Setup One computer will input data and another will be displayed to show notifications on the webapp.

3.2 Testing

Testing is a critical phase in the development of the smart water-level monitor, ensuring that all components function correctly and integrate seamlessly. We will conduct testing at multiple levels, including unit testing for individual modules, integration testing for combined components, and end-to-end testing for the entire system.

3.2.1 Microcontroller

When creating the tests for the microcontroller code, it's important to consider what a user behavior might look like, but also important to understand what level the behavior could get to. How likely is it that a user is putting a full filter on, then an empty one, and back to full, in a matter of milliseconds? How capable is this controller of dealing with sharp spikes and changes? The testing plan covers both simulated behavior for quick and sharp event behavior, as well as a more practical testing plan for typical user behavior.

In order to verify the function of the code reading from the Wheatstone bridge sensor, we must verify that the logic itself works the way we think. There are multiple parts that require logical verification.

First, the sensor data will come in with some small variation depending on where the weight is placed on the base, and because ADC values can vary slightly without any other input. Sensor reading code must also account for the fact that data can change very quickly, so testing the logic through repeated, high frequency changes, can help show its robustness. There is also the issue of whether data inputted to the code is read in correctly through the bit shift operations, so checking the shift operations for correctness is important.

Testing the microcontroller code in this design involves a separate file that contains simulated HX711 behavior. This behavior is passed into the functions used on the microcontroller, which provide an output back to the test file. These outputs are verified against expected outputs and if they are the same or within a certain level of tolerance, then we can say the code passed the tests.

To test the behavior that would be seen for a typical user, we must have standard objects that we use for the different levels of weight. A large weight, a small weight, and no weight are the simplest form of this. For example, fitting with the project, say a water filter with 100mL of water, versus a water filter with 3L of water. These two objects have drastically different weights so putting it on the scale and viewing how the code responds could represent a typical user behavior of taking the empty filter, filling it, and returning it to the base.

3.2.2 Web Application

When testing the web application, we focus on both normal use and edge cases. A normal flow is a user opening the app, calibrating once, and checking the water level. Edge cases include repeated page refreshes, pressing calibration buttons several times, or receiving rapid updates from the sensor. We also test edge cases like if the water level goes beyond expected

limits. We want to ensure the app can handle these scenarios without crashing or showing incorrect information.

For frontend unit tests, we check important UI parts one at a time, such as calibration buttons, status messages, and notification prompts. These tests verify that components render correctly, user clicks update state as expected, and displayed values stay consistent with incoming data. Because sensor readings can change often, we also check that the UI updates smoothly and does not show stale values.

For integration testing, we verify that the frontend and backend communicate correctly. The app sends requests for actions like token registration and sensor data ingestion, and we check that the API returns the expected results. We test both valid and invalid payloads to make sure errors are handled clearly and data is not updated incorrectly.

Finally, end-to-end tests cover complete user workflows from start to finish. These include loading the app, calibrating, monitoring water-level changes, and receiving notifications at threshold events. If these tests pass under both normal and adverse conditions, we can conclude the web application meets functional testing goals.

A Problem Formulation

A.1 Conceptualizations for Design Approach

A.1.1 Contactless Capacitive Water Level Sensing

A contactless sensor is a small puck that attaches to the side of the water filter. It would work by detecting when the level goes below a certain point, and sending that information to the microcontroller. It's a simple and easy way to do one level detection. It produces a binary output of either above or below the detected level.

Pros

- Low-cost
- No contact
- Small mechanical approach

Cons

- Need a good environment for sensor, not humid so not great inside a fridge
- Users need to install device themselves, making it hard to standardize what's considered "empty"
- Binary output doesn't allow for percentage based information, or any form of consumption metrics.

Because of the noted cons to this design style, we decided to find another solution. The user needing to place the sensor on their own device could throw off any configuration we could complete on our side. It puts too much onto the user for it to work well.

A.1.2 Weight based, Load

The design places the pitcher unto a load cell to measure the weight. We will calibrate it to the changes in weight.
Pros

- Highest accuracy of all designs
- Fits to many pitcher dimensions

Cons

- Need a good environment for sensor, not humid so not great inside a fridge

A.1.3 Ultrasonic

An ultrasonic sensor estimates the water level by measuring the echo time of sound waves reflected from the water surface. The sensor would be installed attached to the side of the filter as close to the top of the water compartment.
Pros

- Inexpensive
- Easy implementation

Cons

- Humidity problem
- Mounting hardware and wiring sensor is difficult without modifications to container.

If there are modifications to the filter itself in order for a design to work, then the design could not be possible to produce. If we chose this design and needed to modify the pitcher to mount the hardware or get the wiring right, then we would have to sell pre-modified pitchers as well, otherwise users might not be able to use the device properly. Because of these reasons we decided against using this design style.

A.1.4 Laser

A laser or time-of-flight sensor measures the distance from the sensor to the water surface. The measured distance is converted into liquid height and volume.
Pros

- Continous measurements
- Non contact

Cons

- Humidity problem

- Clear line of sight
- Mounting problems

Some of the same reasons we didn't choose the ultrasonic sensor apply to this laser sensor design. On top of that in order for us to use continuous measurements, the battery power required would raise and we'd have to figure out how to power the device for long periods of time.

A.2 Brainstorming

Items to decide on:

- Microcontroller
- Sensor
  - Ultrasonic: Pointed down from the lid onto a floating object
  - Load Cell: Place the container on the load cell to measure the weight and calculate the water level
  - Laser: Shoots a laser into the water from the lid, and the distance is sent back to the sensor
  - Camera: Setup to watch the water filter and measure the water level based on computer vision
  - Non-Contact Sensor: Patch that sits on the side, when water level goes below sensor it can alert the device
- Server
  - HTTP
  - AWS
  - Vercel
- Challenges
  - Signal in the fridge
  - Size limit
  - Contact Vs No Contact
  - Wifi connectivity
  - Power source (battery, USB, etc.)

A.3 Decision Table

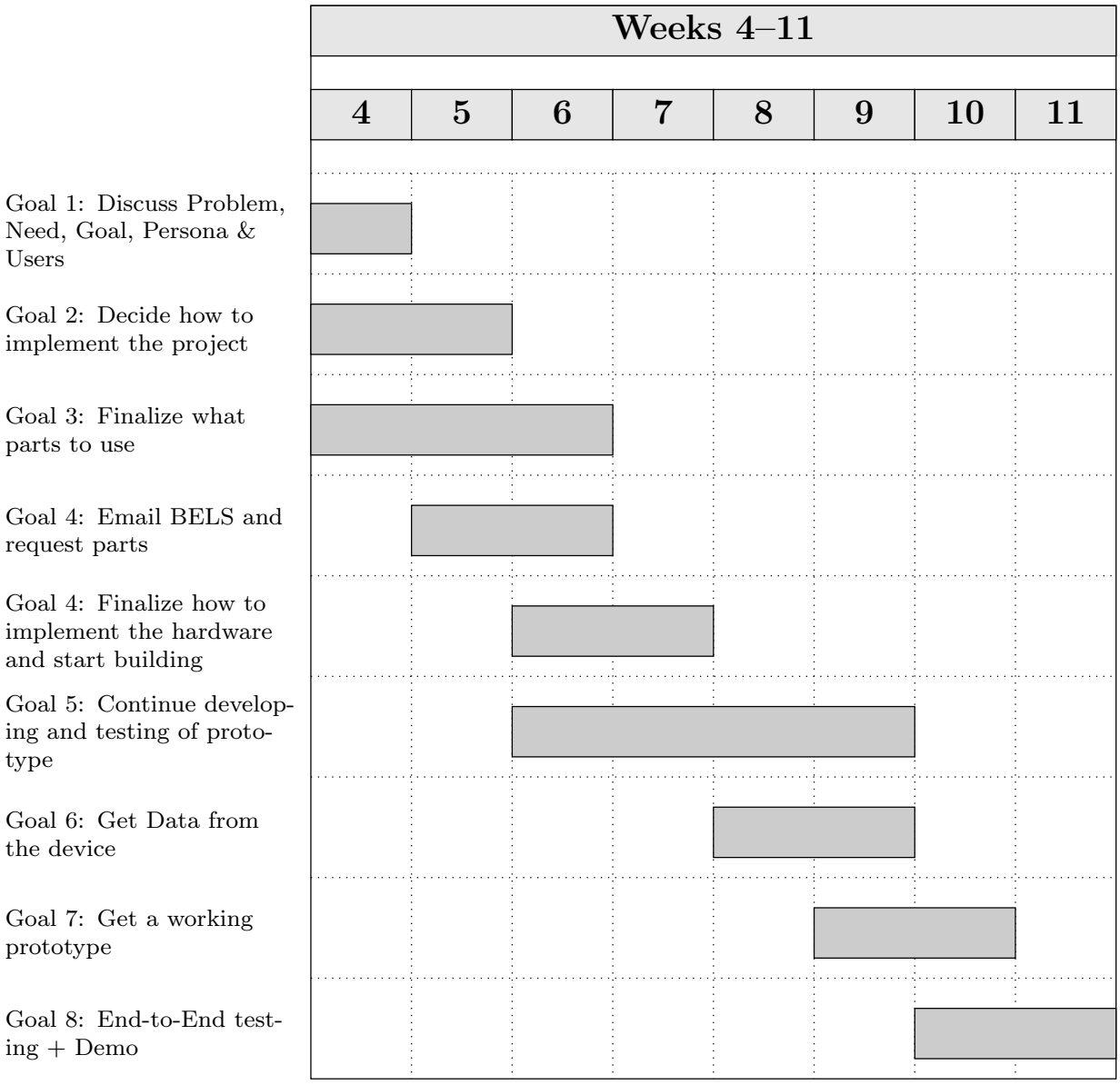
Based on the decision matrix, the weight sensor is the best sensing method because it has the highest weighted total score (4.55). It performs especially well in measurement quality while still keeping cost, power, and overall complexity at a strong level compared to the other options.

Criteria (Weight)	Capacitive	Weight	Laser	Ultrasonic
Cost (0.30)	5	5	3	5
Power (0.20)	5	4	3	4
Complexity (0.25)	3	4	2	3
Measurement Data (0.25)	3	5	3	3
Weighted Total	**4**	**4.55**	**2.70**	**3.85**

Table 1: Decision matrix comparing feasible sensing approaches (1 = poor, 5 = excellent)

B Planning

B.1 Basic Plan / Gantt Chart



B.2 Division of Labor

Dev Chodavadiya: I focused on the physical hardware design and documentation. For hardware, I designed and 3D-printed the base plates for the load cell mount in the BELS lab, iterating on multiple versions to fix flatness and hole alignment issues. I also created the CAD models and engineering drawings for all of the 3D-printed parts. On the documentation side, I contributed to writing and organizing the design document and weekly status reports.

Logan Bossuwe: Handle Load Cell and HX711 sensor hardware to the microntroller. Code weight readings and integrate all microcontroller files.

Pieter Nauwelaerts:

Reid Graham: I mainly worked on the microcontroller code and hardware. I helped with physical assembly of the device, including soldering and plate assembly. On the software side, I implemented the WiFi provisioning system, which allows the device to host a Wifi access point and an HTTP server for users to input their WiFi credentials. I also implemented the HTTP functionality for the microcontroller, which allows it to send weight data to the web application.

In the overall project, I helped to organize our planning using the GitLab repository and the general outline of the design document. I also contributed to the planning the higher level organization of the system through the architecture diagram and organizing the work into discrete modules.

Sam Lai: I mainly worked on the web application development and testing. Specifically, I implemented the notification system that allows users to receive alerts on their laptops and phones when the water level of the water pitcher is low. I created the UI that allows users to subscribe to notifications on the frontend and also created a table in Supabase to store user subscription data. I implemented two API endpoints to handle subscription management and notification sending. Additionally, I assisted Shriyan in testing the overall web application by creating and conducting unit tests for each functional part on the UI to ensure everything works as intended and is reliable. In terms of the design document, I contributed to writing and organizing the web application code and test sections. I also proposed the idea of using a weight sensor and wrote down brainstorming ideas for the design.

Shriyan Gote:

B.3 Collaboration

Using Gitlab, tasks are self assigned on the issue board that are closely related to our division of labor. From our individual contributions, we have weekly meetings for everyone to catchup on what we did in person and online using discord. Whenever individuals tasks have overlap or dependency need we collaborate on the task. Tests plans/code are made in communication with the original maker of a task.

C Test Plan & Results

C.1 Test Plan & Results

There are multiple sections of our design that require testing individually, then together as a whole. The following list represents the pathway we will take when testing our prototype segments.

- ESP Data Reporting
- ESP Web Posting
- Vercel Receiving and Displaying Data
- Notification System
- Phone App Integration
- Prototype Setup Out-Of-The-Box

ESP Data Reporting

The pressure sensors we use to build the base will output data to our ESP device. We want to be sure the ESP is capable of receiving this data and displaying it to the debug terminal, which will be IDF MONITOR. Once we can verify posting to the terminal we will move to the next stage of the data reporting testing.

With data being displayed properly, we can now use this to interpret what an empty versus full pitcher could be expected to weigh. The first test will include to test a known weight to make sure we get accuracate weight readings. Next we will ensure the that when there is no wight on the plate, the data will be read as 0 g. Once this correct, we will move on to the more complex calibration with Pitcher + water.

We will use different types of containers given the variety of containers possible to be used by customers. We will test at different set water levels, and verify that the expected weight represents the following formula.

$$W_{total} = W_{container} + V * \frac{g}{mL} \tag{1}$$

Water weighs 1 gram per milliliter so if we keep track of the volume we put into the container we know exactly what to expect for output weight if we know the base weight of the container alone.

ESP Web Posting

To test this functionality the simplest way is to have the ESP send some sort of HTTP POST pointed at one of our local machines. We can receive and print this data and then move forward to testing sensor data output.

With a working sensor we can trigger a POST request when an event happens. In this test case it could be picking up the container from the base which triggers a POST request

with some set information. Upon successful receipt of this we can confirm that the HTTP method is working on a small scale and move to the next segment of testing.

Vercel Receiving and Displaying Data

Once we have confirmed the ESP can send POST requests to a local machine, the next stage is to implement that using Vercel.

Notification System

With data being sent out of the ESP, the next stage is to test if we can send a notification on some set event. For a test case this could be a container being placed on the device. The data sent as part of this notification could be a value representing the weight of the container, or an estimate of how much liquid is inside.

If we know how much liquid is inside the container when we place it on the base, it is easy to verify if the value we receive in the notification is the correct volume. Using different volumes we can create a table to show how close the reported volume was, and tweak our math to get it closer to accurate with different containers.

When we pass the fake data tests, we will compare the debug terminal and the notification water levels to ensure that it is correct.

Phone App Integration

Prototype Setup Out-Of-The-Box

In order to set up the prototype, the web app must connect to the ESP through its hosted soft AP and scan a QR code to enter the WiFi credentials. To test this, we will try to connect to the ESP on boot multiple times using different devices, such as iOS vs Android, and see if it is still able to connect through soft AP. We will also test different types of WiFi networks, such as mobile hotspots, traditional password-protected networks, and networks with outside authorization such as university WiFi.

The user must also calibrate the device by measuring the empty and full weights of the filter on the weight sensor. On the webapp, they will be given instructions step-by-step to do this. In order to test this process, we will give the device to a new user and see if they are able to clearly follow the instructions and calibrate the device correctly. We will also test robustness by intentionally disobeying the instructions, such as by weighing it empty both times, to see our device handle it.

D Review

D.1 Team Reflections

TODO
Delete this section prob it overlaps too much w the division of labor and wasnt required

Dev Chodavadiya:
Logan Bossuwe:
Pieter Nauwelaerts:
Reid Graham:
Sam Lai:
Shriyan Gote: